COSC3500 - Introduction to OpenMP

Ian McCulloch

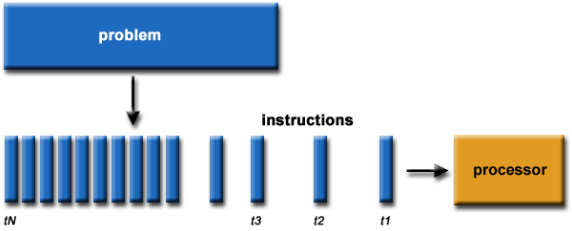
University of Queensland
School of Mathematics and Physics

Semester 2, 2022

What is parallel computing?

In the old days, software was written for **sequential computation**

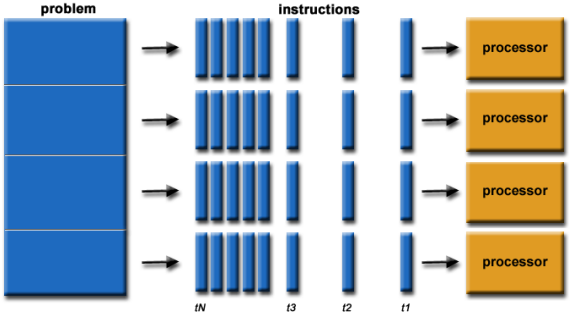
- program consists of a discrete series of instructions
- each instruction is executed sequentially, one after another
- executed on a single processor
- only one instruction is executing at any moment in time
- *note* This is from the *programmer's viewpoint*. Out-of-order execution, prefetch etc give some amount of 'internal' parallelisation but in terms of your program, it runs 'as if' only one instruction is executing at a time.



(image from Blaise Barney, Lawrence Livermore National Laboratory)

Parallel computation is the simultaneous use of multiple execution units to solve a problem:

- program is broken into discrete parts that can be solved concurrently
- different parts are able to execute simultaneously on different processors
- need a mechanism to control/coordinate the computation



(image from Blaise Barney, Lawrence Livermore National Laboratory)

Flavours of parallel processing

There are lots of ways that parallel computation can be controlled.

- **SIMD** – Single Instruction Multiple Data
  One instruction stream that acts on multiple bits of data simultaneously
  - Old-style vector supercomputers – Cray 1 (1976) vector registers containing 64× 64-bit double-precision floating point numbers
  - Re-emergence in the late 90's with Intel MMX, SSE, SSE2, SSE3, SSSE3, SSE4, SSE5, AVX, AVX2, AVX-512 (32 registers, each 64 bytes – close to Cray 1 vector registers!)
  - Compilers will use these instructions automatically, or use hand-tuned assembly language libraries, or compiler-provided *instrinsic* funtions
  - GPU devices are a form of SIMD – one 'kernel' runs on a large number of GPU cores

Flavours of parallel processing

- **MIMD** – Multiple Instruction Multiple Data
  Multiple instruction streams acting independently
  - Hand-rolled threads (eg pthreads, various libraries, C++11 std::thread)
  - Parallel languages – never gained mainstream popularity
  - Built-in support – Java, Julia, C++17
  - Compiler add-ons – OpenMP: supported by many languages

C/C++
FORTRAN

OpenMP is in active development

- Version 1.0 released in 1997, version 5.0 released in November 2018
- current generation compilers support $\sim$ version 4.5
  - GCC 13 has limited support for OpenMP 5.2
- basic MIMD instructions are very mature and stable
- new features are mostly C++ language support, SIMD/GPU instructions

not in this course

What is difficult about parallel programming?

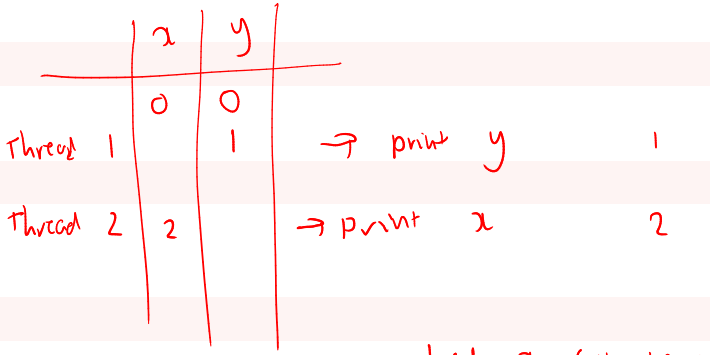
```
int x = 0;
int y = 0;
```

- Thread 1

```
y = x+1;
printf("y=%d\n", y);
```

- Thread 2

```
x = y+1;
printf("x=%d\n", x);
```



load x from memory
add 1 to register
→ save to memory

What are the values of x and y after these two threads have run?

a. $x = 1, y = 2$
b. $x = 2, y = 1$
c. *either* $x = 1, y = 2$ *or* $x = 2, y = 1$, but no way to predict which
d. $x = 1, y = 1$
e. Something else?

Invalid code

OpenMP Exercise 1

Verify that your OpenMP environment works

```
git clone git://git.science.uq.edu.au/cosc3500
cd cosc3500/openmp
make hello_openmp
./hello_openmp
```

Our first OpenMP program:

```
#include <omp.h>
#include <cstdio>

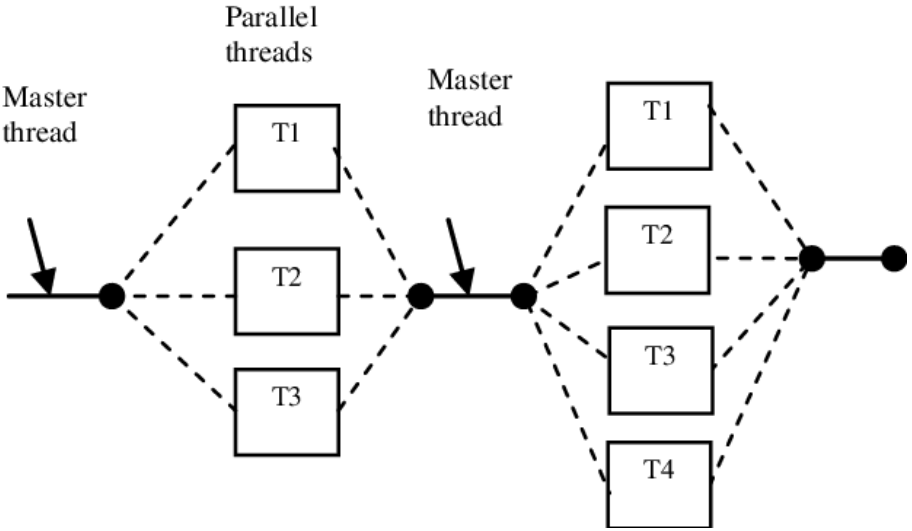
int main()
{
  #pragma omp parallel
   {
      int ID = omp_get_thread_num();
      printf("hello, I am thread number %d ", ID);
      printf("out of %d\n", omp_get_num_threads());
   }
}
```

OpenMP synchronization

- OpenMP is a multi-threading, shared address model
  - Threads communicate by sharing variables
- Unintended sharing of data cause *race conditions*
  - when the program's outcome changes as the threads are scheduled differently
- To control race conditions, use synchronization to prevent data conflicts
- Synchronization is expensive, so design the program to minimise the need for synchronization

OpenMP Programming Model

OpenMP is designed to be able to add parallelism to an existing program



Thread creation: parallel regions

Create threads in OpenMP with the `parallel` construct

Runtime function to request 4 threads

```
double A[1000];
omp_set_num_threads(4);
#pragma omp parallel
{
   int ID = omp_get_thread_num();
   foo(ID, A);
}
```

return the thread ID

Each thread calls `foo(ID)` for ID=0,1,2,3
Shares the same copy of `A`

Thread creation: parallel regions

Create threads in OpenMP with the `parallel` construct

equivalent to previous version

can be constant or variable

```
double A[1000];
#pragma omp parallel num_threads(4)
{
   int ID = omp_get_thread_num();
   foo(ID, A);
}
```

Each thread calls `foo(ID)` for ID=0,1,2,3
Shares the same copy of `A`

Thread creation: parallel regions

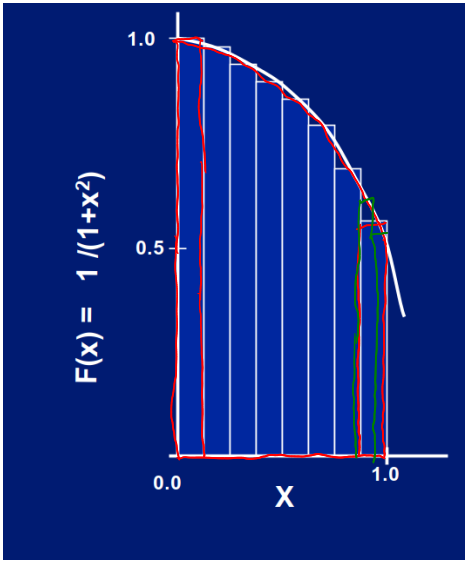
Create threads in OpenMP with the `parallel` construct

```
double A[1000];
#pragma omp parallel
{
    int ID = omp_get_thread_num();
    foo(ID, A);
}
```

control number of threads at runtime with the `OMP_NUM_THREADS` environment variable

```
export OMP_NUM_THREADS=4
./example1
```

Exercise 2 – numerical integration



We are going to write a program to numerically integrate the function

$$\int_0^1 \frac{1}{x^2+1} = \frac{\pi}{4}$$

Numerical approximation: replace the integral with a rectangle which has width Δx and height $F(x_i)$ at the middle of interval i.

pi_serial.cpp

```cpp
#include <cstdio>
#include <cmath>

using namespace std;

int main()
{
   int const num_steps = 100000;
   double const step = 1.0 / num_steps;

   double sum = 0;

   for (int i = 0; i < num_steps; ++i)
   {
      double xi = (i+0.5)*step;
      double f = 1.0 / (xi*xi+1.0);
      sum += f;
   }

   double result = step * sum;

   printf("integral is %.15g\n", result);
   printf("error is %.10e\n", abs(result - M_PI/4));
}
```

$$\frac{1}{x^2+1}$$

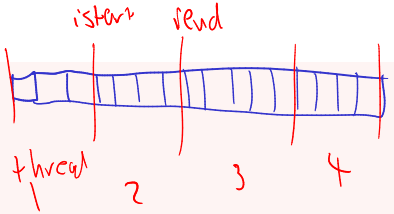
Synchronization

- Basic strategy to parallelize: divide the integral into sections and sum the sections separately
- Need to *synchronize* the result from each thread to obtain the final result

We *could* do this the hard way:

```
int const num_steps = 100000;
#pragma omp parallel
{
   int num_threads = omp_get_num_threads();
   int ID = omp_get_thread_num();
   int istart = ID * num_steps / num_threads;
   int iend = (ID+1) * num_steps / num_threads;
   if (ID == num_threads-1)
      iend = num_steps;

   for (int i = istart; i < iend; ++i)
   {
      // ...
```



Synchronization

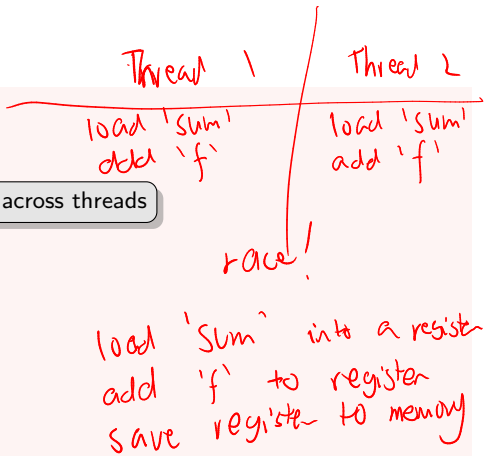
OpenMP will do the book-keeping for us:

```
#include <cstdio>
#include <cmath>
using namespace std;
int main()
{
   int const num_steps = 100000;
   double const step = 1.0 / num_steps;
   double sum = 0;
   #pragma omp parallel
   {
    #pragma omp for
      for (int i = 0; i < num_steps; ++i)
      {
         double xi = (i+0.5)*step;
         double f = 1.0 / (xi*xi+1.0);
         sum += f;
      }
   }
   double result = step * sum;
   printf("integral is %.15g\n", result);
   printf("error is %.10e\n", abs(result - M_PI));
}
```

Split execution across threads

Need Synchronisation.

Catastrophe!



Synchronization

First attempt solution

```
#include <cstdio>
#include <cmath>
using namespace std;
int main()
{
   int const num_steps = 100000;
   double const step = 1.0 / num_steps;
   double sum = 0;
   #pragma omp parallel
   {
      #pragma omp for
      for (int i = 0; i < num_steps; ++i)
      {
         double xi = (i+0.5)*step;
         double f = 1.0 / (xi*xi+1.0);
         #pragma omp critical
            sum += f;
      }
   }
   double result = step * sum;
   printf("integral is %.15g\n", result);
   printf("error is %.10e\n", abs(result - M_PI/4));
}
```

$(a+b)+c \neq a+(b+c)$

$a = 1$

$b = -1$

$c = 10^{-100}$

only 1 thread can run this statement at a time

An efficient solution

```
#include <cstdio>
#include <cmath>
using namespace std;
int main()
{
   int const num_steps = 100000;
   double const step = 1.0 / num_steps;
   double sum = 0;
   #pragma omp parallel
   {
      double local_sum = 0;
      #pragma omp for
      for (int i = 0; i < num_steps; ++i)
      {
         double xi = (i+0.5)*step;
         double f = 1.0 / (xi*xi+1.0);
         local_sum += f;
      }
      #pragma omp critical
         sum += local_sum;
   }
   double result = step * sum;
   printf("integral is %.15g\n", result);
   printf("error is %.10e\n", abs(result - M_PI/4));
}
```

independent

one \ copy per thread

- The result isn't deterministic – why not?

Controlling the number of threads

- From the shell

```
export OMP_NUM_THREADS= <number of threads>
```

- API function:

```
void omp_set_num_threads(int number)
```

- Specify as part of the parallel region

```
#pragma omp parallel num_threads (thread_count)
```

OpenMP doesn't guarantee that this will actually start `thread_count` threads.
Use

```
int omp_get_max_threads();
```

to get an upper bound for the actual number of threads that might be used in a parallel region without an explicit `thread_count`.
You can create a large number of threads if you want, up to some operating-system limit:

```
cat /proc/sys/kernel/threads-max
```

More OpenMP features

- Another shortcut:

```
#pragma omp parallel for
```

  is equivalent to

```
#pragma omp parallel
#pragma omp for
```

- *reduction*
  It is a common operation that multiple threads are calculating some intermediate value which needs to be combined into a final result.

```
double av = 0.0;
#pragma omp parallel for reduction (+:av)
for (int i = 0; i < N; ++i)
{
   av += foo(i);
}
av = av/N;
```

- Exercise: modify the integration example to use a reduction

More OpenMP features – variable scoping

- Variables which are defined in an outer scope are *shared* in parallel regions by default
- This is dangerous! Nothing to prevent accidentally modifying variables in a parallel region
- Better to explicitly mark variables that will be shared between threads

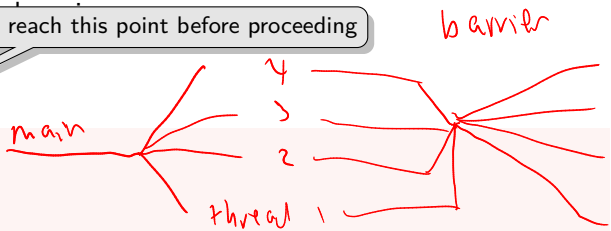
```
#pragma omp parallel default(none), shared(sum), private(f)
```

- Variables marked *private* get a local copy, which is *undefined* on entry to the parallel region
  - Not much reason to use private variables – use a local scope variable instead
- *default(none)* is a good idea - don't share any variables by default
- Compatability problem with *const* variables: in older versions of OpenMP, *const* variables are implicitly shared, even with `default(none)`, and it is an error to share them. Since OpenMP 4.0 and GCC 9 you need to share them.
- Also *threadprivate* scope – each thread gets a copy, but the value is preserved across parallel regions

More synchronization - barriers

- Barrier: each thread

Wait for all threads to reach this point before proceeding




```
#pragma omp parallel shared(A,B,C)
{
   int id = omp_get_thread_num();
   A[id] = big_calculation(id);
   #pragma omp barrier
   #pragma omp for
   for (int i = 0; i < N; ++i)
   {
      B[i] = another_big_calculation(i, A);
   }
   // implicit barrier after worksharing construct
   #pragma omp for nowait
   for (int i = 0; i < N; ++i)
   {
      C[i] = yet_another_big_calculation(i, B);
   }
   A[id] = do_something(B);
}
```

No implicit barrier after this loop

More synchronization - master thread

- *master* construct denotes a block of code that is executed by the master thread. Other threads just skip over it (no synchronization)

```
#pragma omp parallel
{
   do_calculation();
   #pragma omp master
   {
      exchange_boundaries();
   }
   #pragma omp barrier
   do_another_calculation();
}
```

More synchronization - a single thread

- *single* construct denotes a block of code that is executed by only one thread (not necessarily the master)

```
#pragma omp parallel
{
   do_calculation();
   #pragma omp single
   {
      exchange_boundaries();
   }
   // there is an implicit barrier at the end of a single block
   // (can remove it with a nowait clause)
   do_another_calculation();
}
```

More synchronization - multiple critical sections

- If you have multiple critical sections that can run independently, you can *name* them

```
#include <cstdio>
#include <cmath>
using namespace std;
int main()
{
   int x = 0;
   int y = 0;
   #pragma omp parallel
   {
      int ID = omp_get_thread_num();
      int a = foo(1ID);
      #pragma critical (section_1)
      {
         x += a;
      }
      int b = foo2(ID);
      #pragma critical (section_2)
      {
         y += b;
      }
   }
}
```

Summary

- Easy to apply to existing code
  - Identify the part that takes the most CPU time and parallelize that section
- Parallelize outer-most loops
- Watch out for race conditions
- pay attention to shared variables
- use *barriers* and *single* regions for more complex data flows
- Be aware that you might not get the same number of threads that you requested!
  - always use `omp_get_num_threads()`
  - Lots of other features – see resources on Blackboard for more